

Reproducibility Documentation:

All-Atom MD Setup of 4BS2 with 8-Oxoguanosine RNA Modification

Caroline Ge
carolinge73@gmail.com

April 30, 2026 (revision 2)

Abstract

This document is a complete record of every step taken to set up, parameterise, simulate, and analyse the wild-type 4BS2 protein–RNA complex and its 8-oxoguanosine variant. It is written for an investigator with general MD experience but no prior knowledge of this project, with the explicit goal of bit-for-bit reproducibility. Every input file, command line, parameter value, and software version relevant to the published numbers is listed. All custom code is tracked in the repository at `d:/all_atom` (Windows host) and deployed to `/data/biophys/carolinge/clawork/37_OXR/` on the PKS `newton` cluster (referred to as “bio” below).

Contents

1	Software environment	1
1.1	Operating systems and hardware	1
1.2	Conda environment	1
2	Project layout	2
2.1	Workstation (<code>d:/all_atom/</code>)	2
2.2	Cluster (<code>/data/biophys/carolinge/clawork/37_OXR/</code>)	2
3	Source structure: 4BS2	2
4	Force field for 8-oxoguanosine	3
4.1	Atomic changes vs. canonical guanosine	3
4.2	Partial charges	3
4.3	Supplemental bonded parameters	3
4.4	Residue templates	4
4.5	Validation	4
5	Initial structure preparation	4
5.1	Wild-type structure	4
5.2	8OG variant: residue surgery via NeRF	5
5.3	File outputs	5
6	Simulation protocol	6
6.1	Force-field stack	6
6.2	Solvation	6
6.3	Energy minimisation and equilibration	6
6.4	Production	6
6.5	Replica generation	6
6.6	Slurm submission script	7
6.7	Output files per replica	7
6.8	Resuming a failed replica	7

7	Analysis	7
7.1	Single-trajectory binding analysis	8
7.2	Single-trajectory RMSF	8
7.3	Multi-replica analysis with partial-coverage averaging	8
7.4	Plot generation	9
8	Step-by-step reproduction recipe	9
8.1	One-time setup (workstation)	9
8.2	One-time setup (cluster)	9
8.3	Generate starting structures	9
8.4	Generate replica directories	9
8.5	Upload to cluster and submit	9
8.6	Wait, then analyse	10
9	Known issues and gotchas	10

1 Software environment

1.1 Operating systems and hardware

Workstation	Windows 10/11, Python 3.11, used for setup, analysis, plotting
Cluster	<code>newton.pks.mpg.de</code> , Slurm scheduler (CentOS-style RHEL)
GPUs	NVIDIA A100 (mixed precision; CUDA Toolkit 12.5 module)
Cluster home	<code>/home/carolinge</code> ; 80 GB quota
Cluster scratch	<code>/data/biophys/carolinge/</code> (1.4 PB pool, 444 TB free)

Important note on disk layout. The cluster home filesystem (`/home`) has an 80 GB user quota. A single 100 ns trajectory at the parameters used here is ~ 3 GB, so all production output must be written to `/data`. We use a symlink to keep paths consistent:

```
1 ln -s /data/biophys/carolinge/clawork/37_0XR ~/work/37_0XR
```

1.2 Conda environment

The environment name on the cluster is `allatom`. Pinned versions of the load-bearing packages (`conda list`):

<code>python</code>	3.11
<code>openmm</code>	8.2.0
<code>cuda-version</code>	12.5
<code>cuda-nvrtc</code>	12.5.82
<code>pdbfixer</code>	1.10
<code>mdanalysis</code>	2.10.0
<code>numpy</code>	2.x
<code>biopython</code>	1.85
<code>matplotlib</code>	3.9.x

A frozen environment specification is committed to `environment_server.yml` in the repository root.

2 Project layout

2.1 Workstation (`d:/all_atom/`)

```

1 d:/all_atom/
2   app/
3     server.py           # Flask UI for the MD setup pipeline
4     ptm_builder.py      # NeRF-based PTM applicator (defines 8OG patch)
5     ff_custom/
6       8OG_RNA_amber.xml # Custom force field for 8-oxoguanosine
7     templates/index.html # Web UI
8   scripts/
9     gen_4bs2_md.py      # Generate WT and 8OG starting structures
10    gen_replicas.py      # Generate 3 WT + 3 8OG replica directories
11    analyze_binding.py   # Single-trajectory binding analysis
12    analyze_rmsf.py      # Single-trajectory per-residue RMSF
13    analyze_replicas.py  # Multi-replica analysis with partial-cov
14      ↪ averaging
15    plot_rmsf.py         # Plot RMSF comparison
16    compare_wt_8og.py    # Plot WT vs 8OG comparison panels
17  output/
18    4BS2_WT_MD/          # 10 ns preliminary WT run
19    4BS2_8OG_G3_MD/      # 10 ns preliminary 8OG run
20    4BS2_WT_r{1,2,3}/    # Local staging of 100 ns replica directories
21    4BS2_8OG_r{1,2,3}/
22    analysis/            # All analysis CSVs and PNGs
23  docs/
24    4BS2_8OG_report.tex  # Main results report
25    4BS2_8OG_methods.tex # This file

```

2.2 Cluster (/data/biophys/carolinge/clawork/37_OXR/)

```

1 /data/biophys/carolinge/clawork/37_OXR/
2   04_4BS2_WT_MD/        # 10 ns preliminary WT run
3   05_4BS2_8OG_G3_MD/    # 10 ns preliminary 8OG run
4   replicas/
5     4BS2_WT_r1/ ... _r3/ # 100 ns WT replicas (3 GB DCD each)
6     4BS2_8OG_r1/ ... _r3/ # 100 ns 8OG replicas
7   anal_gyrate/          # All analysis output: CSVs, NPZs, PNGs
8   analyze_binding.py     # uploaded copies of the local scripts
9   analyze_rmsf.py
10  analyze_replicas.py

```

3 Source structure: 4BS2

The structure used as the starting point is PDB entry **4BS2** (Lukavsky et al. 2013; an RRM–RNA complex). The CIF file (d:/all_atom/4BS2.cif) was downloaded from the RCSB PDB and contains:

- Chain A: 174 amino acids (the RRM domain, GLY1–GLN174);
- Chain B: 12 nucleotides of single-stranded RNA with sequence GUGUAGAAUGAAU.

The third nucleotide on chain B (G3) is the target of the 8-oxo modification.

4 Force field for 8-oxoguanosine

The wild-type RNA force field is AMBER OL3 (Pérez et al. 2007, amber14/RNA.OL3.xml as shipped with OpenMM 8.2). 8OG requires a custom XML (app/ff_custom/8OG_RNA_amber.xml) that *augments*, not replaces, OL3. The two XMLs are loaded together when the system is built (Section 6).

4.1 Atomic changes vs. canonical guanosine

Atom	Change	Notes
H8	deleted	aromatic CH on C8 in G is removed
C8	atom type RNA-CP $\rightarrow$ RNA-C	sp ² aromatic $\rightarrow$ sp ² carbonyl C
N7	atom type RNA-NB $\rightarrow$ RNA-NA	aromatic N $\rightarrow$ sp ² N-H
O8	added (RNA-O)	C8=O8 carbonyl oxygen
H7	added (RNA-H)	N7-H proton (donor)

No new atom types are introduced: RNA-C, RNA-O, RNA-NA, RNA-H, RNA-CB, RNA-N* are all defined in `amber14/RNA.OL3.xml`. Only previously unused *combinations* of these types appear in 8OG, which is why new bond/angle/torsion entries are needed (Section 4.3).

4.2 Partial charges

Backbone and sugar charges are copied verbatim from the standard G residue in OL3.xml. Base charges are unchanged from G except in the N7-C8-O8-H7 region. Modified charges (in elementary charge units):

Atom	G (OL3)	8OG (this work)
C8	+0.1374 (RNA-CP)	+0.4000 (RNA-C, carbonyl)
H8	+0.1640 (deleted)	—
N7	-0.5709 (RNA-NB)	-0.4600 (RNA-NA)
H7	—	+0.3500 (RNA-H)
O8	—	-0.5600 (RNA-O)

Reference and limitations. Values are derived by analogy from [Aduri et al. \(2007\)](#), who report RESP charges for several modified RNA nucleosides. The exact 8OG charges in their work are for the DNA-context 8OG; for the RNA context we kept the OL3 backbone (which already differs from the Aduri scheme) and only redistributed charge locally around C8/N7. **Net residue charge is preserved at -1.0** (the same as canonical phosphorylated G):

$$\Delta q_{\text{base}} = (-0.1640) + (+0.2626) + (+0.1109) + (+0.3500) + (-0.5600) = -0.0005 \approx 0. \quad (1)$$

For publication-quality work this charge model should be replaced with a fresh QM RESP fit using the Antechamber pipeline. The validation script (`scripts/validate_8OG_ff.py`) confirms that the present charges and topology load cleanly into OpenMM and produce a finite-energy system, but does not validate accuracy.

4.3 Supplemental bonded parameters

OL3 lacks parameters for the type-pair RNA-CB-RNA-NA because canonical guanine has RNA-NB at N7. Five bonded terms are added by analogy to existing OL3 entries:

```

1 <HarmonicBondForce>
2   <!-- C5(CB)-N7(NA) bond, derived from CB-NB in OL3.xml: k=346435, r=0.1391 nm -->
3   <Bond k="346435.2" length="0.1390"
4     type1="RNA-CB" type2="RNA-NA"/>
5 </HarmonicBondForce>
6
7 <HarmonicAngleForce>
8   <!-- C8-N7-C5 angle in 5-membered imidazole ring (~103.8 deg = 1.8117 rad)
9     analogous to RNA-CB-RNA-NB-RNA-CP in OL3 -->
10  <Angle angle="1.8117" k="585.76"
11    type1="RNA-CB" type2="RNA-NA" type3="RNA-C"/>
12
13  <!-- C5-N7-H7 exocyclic angle (~139.4 deg = 2.433 rad).
14    Derived from planarity: 360 - 103.8 (C8-N7-C5) - 116.8 (C8-N7-H7) -->

```

```

15 <Angle angle="2.433" k="418.40"
16     type1="RNA-CB" type2="RNA-NA" type3="RNA-H"/>
17 </HarmonicAngleForce>
18
19 <PeriodicTorsionForce ordering="amber">
20 <!-- X-CB-NA-X torsion, analogous to X-CB-NB-X in OL3 -->
21 <Proper k1="10.6692" periodicity1="2" phase1="3.141592653589793"
22     type1="" type2="RNA-CB" type3="RNA-NA" type4=""/>
23
24 <!-- N* improper to enforce planarity at N9 (analogous to G in OL3,
25     updated for the new C8 atom type) -->
26 <Improper k1="4.184" periodicity1="2" phase1="3.141592653589793"
27     type1="RNA-N*" type2="RNA-CB" type3="RNA-C" type4="RNA-CT"/>
28 </PeriodicTorsionForce>

```

Listing 1: Supplemental bonded parameters in 8OG_RNA_amber.xml

All other 8OG bonded interactions use parameters that already exist in OL3 for canonical RNA atom-type pairs.

4.4 Residue templates

Three residue templates are defined to handle the three positional contexts of 8OG in an RNA chain:

- 8OG – internal residue, `ExternalBond` on both P (5' side) and O3' (3' side).
- 8OG5 – 5'-terminal: no phosphate; H05' caps O5'; only O3' forms an external bond.
- 8OG3 – 3'-terminal: no external bond on O3'; H03' cap added.

4.5 Validation

The script `scripts/validate_8OG_ff.py` loads OL3 + the custom XML, builds a 5'-terminal 8OG mononucleotide topology, and calls `ff.createSystem()`. Failure of this step (for example, after introducing a typo in the XML) indicates that the parameter set is incomplete; success indicates that *at least* the bonded topology is consistent.

5 Initial structure preparation

The full preparation pipeline is encapsulated in the script `scripts/gen_4bs2.md.py`; the key steps are summarised here so the reader can recreate them by hand if needed.

5.1 Wild-type structure

1. Load `4BS2.cif` via `PDBFixer`.
2. Call `findMissingResidues()` (no terminal trimming).
3. Call `findMissingAtoms()` and `addMissingAtoms()`.
4. Call `addMissingHydrogens(pH=7.0)`.
5. Write the result to `4BS2_prepared.pdb` (3164 heavy + H atoms, single MODEL).

5.2 8OG variant: residue surgery via NeRF

The 8OG variant is produced from the prepared WT PDB by an idealised geometric procedure implemented in `app/ptm_builder.py`. The procedure for chain B residue 3 is:

1. Read the WT PDB with BioPython.
2. Locate the target residue (chain B, resseq 3, resname G).
3. Delete atom H8 from the residue.
4. Generate the position of O8 using the Natural Extension Reference Frame (NeRF) algorithm with three reference atoms (N9, N7, C8) and ideal internal coordinates:
 - Bond C8--O8: 1.230 Å
 - Angle N7--C8--O8: 128.8°
 - Dihedral N9--N7--C8--O8: 180°(O8 trans to N9)
5. Generate the position of H7 from references (C8, C5, N7):
 - Bond N7--H7: 1.010 Å
 - Angle C5--N7--H7: 125.2°
 - Dihedral C8--C5--N7--H7: 180°(H7 in imidazole plane)
6. Rename the residue to 8OG (or 8OG3/8OG5 if it is the 3' or 5' terminus).
7. Write the modified structure to PDB.
8. Append CONECT records for all intra-residue bonds of 8OG *and* for the inter-residue O3'–P backbone bonds spanning the modified residue. This is essential because OpenMM's `createStandardBonds()` routine builds bonds only for residues named in its built-in standard table; for any non-standard name it silently skips both the residue's internal topology *and* its inter-residue backbone bonds. Without the explicit CONECT records, `ForceField.createSystem()` fails with the misleading message “*atoms match 8OG, but the bonds are different*”.

NeRF algorithm. The NeRF subroutine takes three known atom positions **a**, **b**, **c** and three internal coordinates (bond length b , bond angle θ , dihedral ϕ), and returns the position of a new atom **d** such that $|\mathbf{c} - \mathbf{d}| = b$, $\angle(\mathbf{b}, \mathbf{c}, \mathbf{d}) = \theta$, and $\text{dihedral}(\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}) = \phi$. The implementation in `ptm_builder.py` is a 7-line numpy function that constructs the local frame at **c**.

Reproduction.

```
1 python -c "
2 import sys; sys.path.insert(0, 'd:/all_atom/app')
3 from ptm_builder import apply_ptm
4 pdb = apply_ptm('output/4BS2_WT_MD/4BS2_prepared.pdb',
5                 chain_id='B', resnum=3, ptm_code='8OG')
6 open('output/4BS2_8OG_G3_MD/4BS2_8OG_G3.pdb', 'w').write(pdb)
7 "
```

5.3 File outputs

After preparation, two files are used as the starting structure of all subsequent simulations:

<code>output/4BS2_WT_MD/4BS2_prepared.pdb</code>	3164 atoms; canonical RRM–RNA
<code>output/4BS2_8OG_G3_MD/4BS2_8OG_G3.pdb</code>	3165 atoms; G3 → 8OG (–H8 +O8 +H7)

6 Simulation protocol

The simulation protocol is identical for the preliminary 10 ns runs (Section ??) and the production 100 ns multi-replica runs (Section 6.4); only the production length and the random seed differ.

6.1 Force-field stack

```

1 ff = ForceField('amber14/protein.ff14SB.xml',      # protein
2               'amber14/RNA.OL3.xml',              # RNA
3               'amber14/tip3p.xml',                 # water
4               'app/ff_custom/8OG_RNA_amber.xml')    # 8OG (only for variant)

```

6.2 Solvation

Modeller.addSolvent with:

- Padding: 1.0 nm (cubic box; box size scales with structure)
- Ionic strength: 0.15 M
- Cation: Na⁺; anion: Cl[−]
- Total system: ~51 407 atoms (WT) / ~51 408 (8OG, +1 atom)

6.3 Energy minimisation and equilibration

1. Energy minimisation: minimizeEnergy(maxIterations=1000).
2. NVT equilibration: 100 ps. LangevinMiddleIntegrator (300 K, $\gamma = 1 \text{ ps}^{-1}$, dt = 4 fs, hydrogen mass repartitioning to 1.5 amu). PME with 1.2 nm cutoff. HBonds constraints. State logged every 500 steps to nvt.log.
3. NPT equilibration: 100 ps. As above, plus MonteCarloBarostat (1 atm, 300 K). State logged every 500 steps to npt.log.

6.4 Production

- Length: 100 ns (25 000 000 steps at 4 fs).
- Trajectory: DCDReporter('traj.dcd', 5000), i.e. a frame every 20 ps, ~5000 frames per replica, ~3.0 GB per file.
- State log: every 5000 steps to md.log.
- Final checkpoint saved to final.chk.
- Hardware: NVIDIA A100, mixed precision; speed ~470 ns/day, wall-clock 5 h per 100 ns replica.

6.5 Replica generation

The script scripts/gen_replicas.py generates 6 directories (3 WT + 3 8OG), each with its own run_openmm.py and sub.sh. Replicas differ in the integer seed (1, 2, ..., 6) which is used at three places to ensure independent samples:

```

1 random.seed(seed)                                # Python RNG, used by Modeller
2 integrator.setRandomNumberSeed(seed)              # Langevin thermostat noise
3 sim.context.setVelocitiesToTemperature(300*K, seed) # initial velocities
4 barostat.setRandomNumberSeed(seed)                 # MC barostat

```

6.6 Slurm submission script

Listing 2: sub.sh (one per replica)

```

1  #!/bin/bash
2  #SBATCH --time=24:00:00
3  #SBATCH --partition=graphic
4  #SBATCH --constraint="gpu"
5  #SBATCH --gres=gpu:a100:1
6  #SBATCH --cpus-per-task=10
7  #SBATCH --mem=225000
8  #SBATCH --job-name="4BS2_<system>_r<n>"
9  #SBATCH --output=./%j.out
10 #SBATCH --error=./%j.err
11
12 set -e
13 module purge
14 module load cuda/12.5
15 source "$(conda info --base)/etc/profile.d/conda.sh"
16 conda activate allatom
17 python ./run_openmm.py

```

To submit one replica:

```

1  cd /data/biophys/carolinge/clawork/37_0XR/replicas/4BS2_WT_r1
2  sbatch sub.sh

```

6.7 Output files per replica

After a successful run, the directory contains:

4BS2_<sys>.pdb	starting structure (input)
8OG.RNA_amber.xml	custom FF (only for 8OG variant)
run_openmm.py	simulation driver (input)
sub.sh	submission script (input)
solvated.pdb	solvated topology written before MD; matches atom order; required for analysis
nvt.log, npt.log	equilibration state logs
md.log	production state log; contains progress % analysis script
traj.dcd	production trajectory (5000 frames, ~3 GB)
final.chk	checkpoint at end of production

6.8 Resuming a failed replica

A replica that died mid-production (for example because of a filesystem-quota error) cannot be resumed mid-run because no intermediate checkpoint is written. Restart from scratch:

```

1  cd /data/biophys/carolinge/clawork/37_0XR/replicas/4BS2_8OG_r3
2  rm -f traj.dcd md.log nvt.log npt.log solvated.pdb final.chk *.out *.err
3  sbatch sub.sh

```

7 Analysis

All analysis scripts are kept under `scripts/` on the workstation and are deployed to the cluster's project root for execution against trajectories on `/data`.

7.1 Single-trajectory binding analysis

`scripts/analyze_binding.py` computes, on a per-frame basis: RNA RMSD vs. frame 0; total heavy-atom contacts ($< 4 \text{ \AA}$) between protein and RNA; protein–RNA centre-of-mass distance; G3-specific contacts; G3 base polar contacts ($< 3.5 \text{ \AA}$, proxy for hydrogen bonds). It also accumulates a per-residue contact-frequency map. Inputs: a topology PDB (must match the DCD atom count) and a trajectory file. Outputs: `<prefix>.cv.csv`, `_contact_map.csv`, `_summary.txt` and PNG plots.

The topology PDB must be the `solvated.pdb` written by `run_openmm.py`, not the unsolvated starting file — the DCD contains all 51 407+ atoms.

7.2 Single-trajectory RMSF

`scripts/analyze_rmsf.py` aligns on protein C α and computes per-residue heavy-atom RMSF for both chains.

7.3 Multi-replica analysis with partial-coverage averaging

`scripts/analyze_replicas.py` is the main workhorse for the 100 ns multi-replica analysis. Key features:

- Accepts an arbitrary number of `--label / --replicas` pairs.
- Loads each replica's `solvated.pdb` + `traj.dcd` independently. Trajectories of unequal length are handled by NaN-padding when stacked, so that at each time-point the cross-replica mean and SEM are computed only over replicas that have actually reached that time-point.
- Applies a *minimum-image PBC correction* relative to the protein centre of mass before computing RNA RMSF and RMSD; this is necessary because RNA atoms can wander to a periodic image, which would otherwise inflate RMSF artifactually.
- Output files: `multi_per_replica.json` (per-replica means), `multi_timeseries.npz/.png`, `multi_rmsf.npz/` and `multi_summary.txt`.

Avoiding the active-trajectory race condition. Reading a DCD file that is concurrently being written by an active MD job will cause the analysis script to die silently mid-frame. The analysis must therefore exclude any replica whose simulation has not yet finished. A simple check is whether the corresponding Slurm job is still in the queue:

```
1 squeue -u carolinge | grep 4BS2
```

Only fully completed replicas should be passed to `analyze_replicas.py`.

Reproduction.

```
1 cd /data/biophys/carolinge/clawork/37_OXR
2 nohup conda run -n allatom python analyze_replicas.py \
3   --label WT --replicas replicas/4BS2_WT_r1 \
4     replicas/4BS2_WT_r2 \
5     replicas/4BS2_WT_r3 \
6   --label 80G --replicas replicas/4BS2_80G_r1 \
7     replicas/4BS2_80G_r2 \
8     replicas/4BS2_80G_r3 \
9   --out anal_gyrate/multi --stride 5 \
10  > /tmp/multi.log 2>&1 &
```

The `--stride 5` option subsamples every 5th frame from each trajectory (1000 effective frames per replica) and reduces wall time to ~ 5 minutes for 6 replicas.

7.4 Plot generation

`scripts/compare_wt_8og.py` (10 ns single-replica figures) and `scripts/plot_rmsf.py` (single-replica RMSF) read the CSVs/JSON emitted by the analysis scripts and write the corresponding PNGs into `output/analysis/`.

8 Step-by-step reproduction recipe

This section is a single-command-per-step recipe to recreate every file from scratch.

8.1 One-time setup (workstation)

```
1 # 1. Clone repository (already in place at d:/all_atom)
2 # 2. Create conda environment
3 conda env create -f environment_server.yml -n allatom
4
5 # 3. Verify the custom force field loads
6 conda run -n allatom python scripts/validate_8OG_ff.py
```

8.2 One-time setup (cluster)

```
1 ssh bio
2 # 4. Mirror the conda environment
3 conda env create -f environment_server.yml -n allatom
4 mkdir -p /data/biophys/carolinge/clawwork/37_OXR
5 ln -s /data/biophys/carolinge/clawwork/37_OXR ~/work/37_OXR
```

8.3 Generate starting structures

```
1 # Workstation
2 cd d:/all_atom
3 conda run -n allatom python scripts/gen_4bs2_md.py
4 # Outputs:
5 #   output/4BS2_WT_MD/4BS2_prepared.pdb
6 #   output/4BS2_8OG_G3_MD/4BS2_8OG_G3.pdb (CONNECT records included)
```

8.4 Generate replica directories

```
1 conda run -n allatom python scripts/gen_replicas.py
2 # Outputs 6 directories under output/4BS2_{WT,8OG}_r{1,2,3}/
```

8.5 Upload to cluster and submit

```
1 # Workstation -> cluster
2 scp -r output/4BS2_*_r? bio:/data/biophys/carolinge/clawwork/37_OXR/replicas/
3 ssh bio
4 cd /data/biophys/carolinge/clawwork/37_OXR/replicas
5 for d in 4BS2_*_r*; do
6     dos2unix $d/sub.sh $d/run_openmm.py 2>/dev/null
7     ( cd $d && sbatch sub.sh )
8 done
9 squeue -u $USER
```

8.6 Wait, then analyse

```

1 # Wait until squeue -u $USER returns no 4BS2_* jobs.
2 cd /data/biophys/carolinge/clawork/37_OXR
3 nohup conda run -n allatom python analyze_replicas.py \
4 --label WT --replicas replicas/4BS2_WT_r{1,2,3} \
5 --label 8OG --replicas replicas/4BS2_8OG_r{1,2,3} \
6 --out anal_gyrate/multi --stride 5 \
7 > /tmp/multi.log 2>&1 &
8
9 # Download outputs to workstation:
10 scp 'bio:/data/biophys/carolinge/clawork/37_OXR/anal_gyrate/multi_*' \
11 d:/all_atom/output/analysis/
12
13 # Compile the report
14 cd d:/all_atom/docs
15 pdflatex -interaction=nonstopmode 4BS2_8OG_report.tex
16 pdflatex -interaction=nonstopmode 4BS2_8OG_report.tex # citations

```

9 Known issues and gotchas

1. **Disk quota on /home.** 80 GB user quota; a single 100 ns trajectory is ~3 GB. All production output must live on /data. Failure to do so manifests as a Slurm job that exits without an error message after ~40 % of production. The symlink trick in Section 1 keeps the path ~/work/37_OXR usable while storing data on /data.
2. **Race condition on active trajectories.** analyze_replicas.py will silently die mid-frame if it reads a DCD that is being concurrently written by an active MD job. Always verify with squeue that all replicas are complete before running the multi-replica analysis. (The script uses NaN-padding to handle replicas of *differing* lengths, but only after they have finished writing.)
3. **CONNECT records are essential for non-standard residues.** OpenMM’s createStandardBonds() bond-detection routine skips any residue whose name is not in its built-in standard table. For 8OG this means *none* of the residue’s intra-residue bonds will be created, and the inter-residue O3’-P backbone bonds spanning the modified residue will also be silently dropped. The result is an opaque error message at ForceField.createSystem() time: “atoms match 8OG, but the bonds are different”. The ptm_builder.py script writes explicit CONNECT records for both the intra-residue bonds (Section 5.2) and the inter-residue backbone bonds (function _add_backbone_connect). Do not modify the output PDB without preserving these CONNECT records.
4. **Windows-style line endings break Slurm.** Submission scripts edited or copied via Windows utilities can pick up CRLF line endings, which cause sbatch to reject the script with a “DOS line breaks” error. Run dos2unix on every *.sh and *.py after upload.
5. **CUDA driver / PTX version mismatch.** \$ nvidia-smi on the login node may report a different driver version than what is installed on the GPU compute nodes. If Platform.getPlatformByName(‘CUDA’) succeeds but a subsequent Simulation() call fails with “CUDA_ERROR_UNSUPPORTED_PTX_VERSION”, fall back to OpenCL or CPU automatically (the run_openmm.py template includes such a fallback loop).
6. **PBC unwrapping for RNA RMSF.** A naive RMSF computation will report values of ~25–40 Å for any RNA atoms that hop to a periodic image. analyze_replicas.py applies a minimum-image correction relative to the protein centre of mass before storing positions. Without this correction, RNA RMSF is uninterpretable.

References

- Aduri R., Psciuk B.T., Saro P., Taniga H., Schlegel H.B., SantaLucia J. AMBER force field parameters for the naturally occurring modified nucleosides in RNA. *J. Chem. Theory Comput.* **3**, 1464–1475 (2007).
- Lukavsky P.J., Daujotyte D., Tollervey J.R., Ule J., Stuani C., Buratti E., Baralle F.E., Damberger F.F., Allain F.H.-T. Molecular basis of UG-rich RNA recognition by the human splicing factor TDP-43. *Nat. Struct. Mol. Biol.* **20**, 1443–1449 (2013).
- Pérez A., Marchán I., Svozil D., Šponer J., Cheatham T.E., Laughton C.A., Orozco M. Refinement of the AMBER force field for nucleic acids: improving the description of α/γ conformers. *Biophys. J.* **92**, 3817–3829 (2007).